

AI Loan Eligibility Assessment Assistant

Problem Statement: Banks manually review loan applications, slowing approvals and introducing inconsistency.

Prompt for Synthetic data loan_application csv file

Role - Act as a financial data generator. I need a synthetic dataset of exactly 50 loan applications formatted as a raw CSV.

Context- The dataset should have these 13 columns: application_id (APP001 to APP050), customer_name (common Indian names), age (26-50), monthly_income, employment_type, credit_score (500-830), existing_loans (Yes/No), loan_amount_requested, loan_purpose, repayment_history, document_status, risk_level, and eligibility_status.

Instructions - Please split the 50 rows into three specific groups based on these rules:

1. 24 'ELIGIBLE' rows: Salaried employees with high income (68k–100k), high credit scores (740–830), no existing loans, 'Good' repayment history, 'Complete' documents, and 'LOW' risk. They are requesting 300k–850k for Home, Car, or Education loans.
2. 13 'REVIEW' rows: Mostly Business owners with mid-tier income (45k–56k), moderate credit scores (600–650), existing loans (Yes), 'Average' repayment history, 'Complete' documents, and 'MEDIUM' risk. They want 900k–1.3M for Business or Renovation loans.
3. 13 'NOT_ELIGIBLE' rows: Mostly Self-Employed individuals with low income (25k–40k), low credit scores (500–590), existing loans (Yes), 'Poor' repayment history, 'Pending' documents, and 'HIGH' risk. They want 500k–1.5M for Personal or Medical emergencies.

Guardrails - Just output the clean, raw CSV data. Please do not include any introductory or concluding text.

Prompt for Synthetic data past_loans csv file

Role - Act as a data generator.

Context - I am testing a new loan processing system and need a realistic mock dataset of 50 Indian applicants to test our logic.

Instructions - Generate a 50-row CSV dataset. Keep the logic realistic: applicants with high credit scores and good histories should be marked as **LOW** risk and **ELIGIBLE**. Applicants with low scores and poor repayment history should be **HIGH** risk and **NOT_ELIGIBLE**. Use realistic Indian names, varied ages, and logical income/loan amounts.

Example - ``csv

```
application_id,customer_name,age,monthly_income,employment_type,credit_score,existing_loans,loan_amount_requested,loan_purpose,repayment_history,document_status,risk_level,eligibility_status APP001,Rahul Sharma,29,85000,Salaried,780,No,500000,Home Loan,Good,Complete,LOW,ELIGIBLE APP002,Priya Verma,41,45000,Salaried,620,Yes,1200000,Business Loan,Average,Complete,MEDIUM,REVIEW APP003,Ahmed Khan,35,25000,Self-Employed,540,Yes,800000,Personal Loan,Poor,Pending,HIGH,NOT_ELIGIBLE
```

Prompt for Legal Policy Guidelines

Role & Context: You are a senior risk manager at a bank writing the initial draft for our new lending policy guidelines.

Instructions: Write a simple, numbered list of 6 to 7 foundational rules for assessing loan applicants. Keep each rule to just one concise sentence.

Make sure to cover:

- Credit score thresholds (like 750)
- Income requirements for large loans
- What triggers a manual review or a temporary hold
- Business loans and having multiple active loans

Tone & Style: Professional, direct, and completely free of corporate fluff.

Prompt for Loan Application PDF

Context: I need to test our loan approval system with a perfect, low-risk applicant profile.

Instructions: Create a mock loan application form for an individual named Anjali Desai. Organize the data into three clear sections: Applicant Info, Loan Details, and Declaration/Documents.

Include the following specific details:

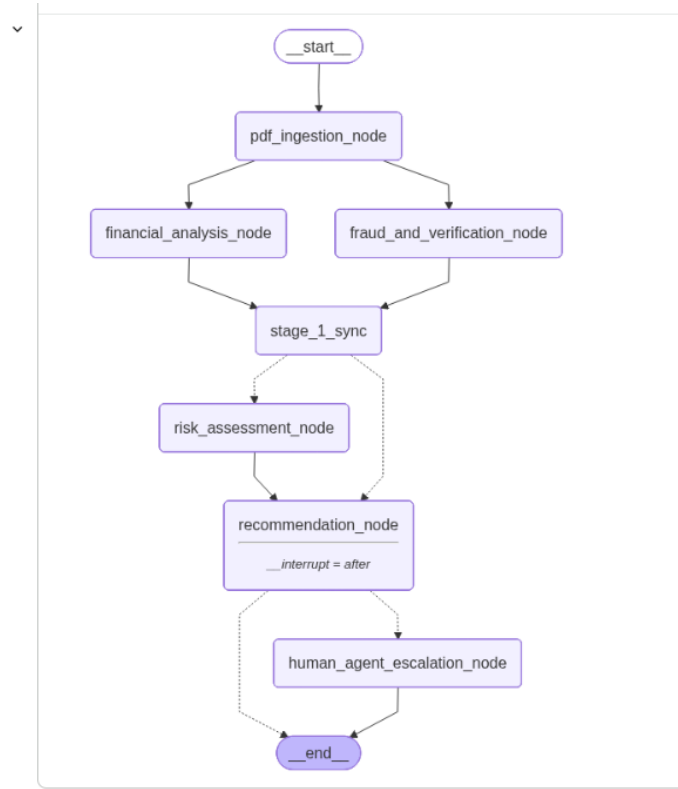
- She is a 32-year-old salaried Software Engineer making 1.2 Lakh a month.
- She has a great credit score (760) and no other active loans.
- She is asking for a 15 Lakh Home Loan Renovation.
- Note that all her required documents (ID, salary slips, bank statements) are attached and successfully verified.

Output Format: Keep it structured with clear section headers, clean labels, and simple values.

Tech stack

Agent Framework	LangGraph (for building the state graph and workflow routing) + LangChain (core messages and tool binding)
LLM	Gemini (gemini-2.5-flash via langchain-google-genai)
RAG	LangChain (using RecursiveCharacterTextSplitter, PyPDFLoader, and custom vector store retrieval tools)
Vector DB	ChromaDB (persistent local vector client storage)

Architecture



Workflow Pipeline Stages

1. Ingestion Layer

- `pdf_ingestion_node`: Serves as the entry point for parsing application PDFs. It verifies essential documentation like ID Proof and Salary Slips, tagging the state as `Complete` or `Pending`.

2. Parallel Processing Layer (Fan-Out)

The workflow diverges into two concurrent tracks to maximize processing efficiency:

- `financial_analysis_node`: Handles computational tasks, specifically estimating debt levels and calculating Debt-to-Income (DTI) metrics.
- `fraud_and_verification_node`: Conducts background validation by analyzing risk patterns and assigning a fraud probability score between `0.0` and `1.0`.

3. Synchronization & Smart Routing (Fan-In)

- `stage_1_sync`: Merges parallel outputs from the verification tracks back into a unified operational state.
- `router_after_sync` (Conditional Gate):
 - If document status is `Pending`, the flow bypasses risk assessment and moves directly to a hold status.
 - If status is `Complete`, the process moves forward to the deep risk analyzer.

4. Evaluation & Retrieval-Augmented Generation (RAG)

- `risk_assessment_node`: The primary logic engine utilizing ChromaDB to query two vector store tools:
 - *Tool 1* (`search_rules_and_policies`): Aligns credit scoring with established policy guidelines.
 - *Tool 2* (`find_similar_historical_applications`): References past applications to maintain decision consistency.

5. Interruption Layer & Human Verification

- `recommendation_node`: Formulates the specific approval terms.
- State Interruption (`__interrupt = after`): The graph freezes to allow manual oversight. State is preserved via `MemorySaver`, resuming only when a human provides an `accept` or `escalate` input.

6. Resolution Layer

- `router_after_review` (Conditional Gate): Approval routes the state to `__end__`. Escalation triggers the `human_agent_escalation_node` to create a manual ticket for a bank representative.